



FlowNote - AI 기반 PARA 분류 시스템

[한국어](#) | [English](#)

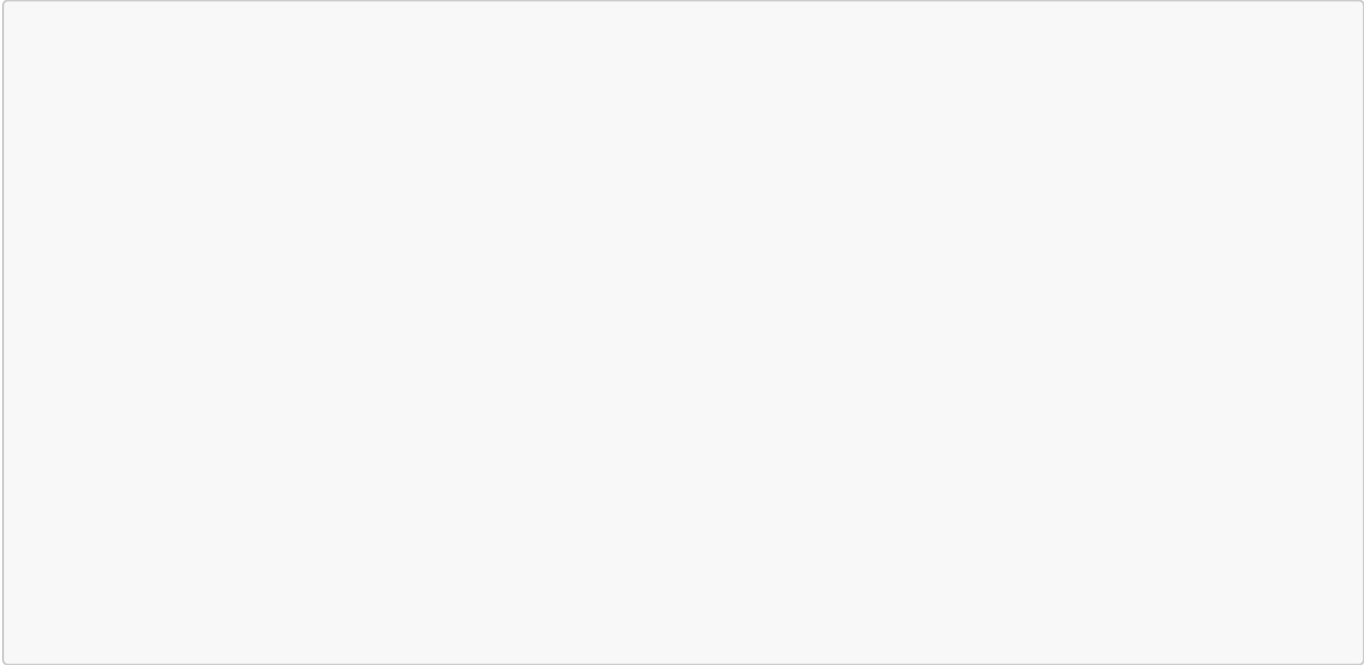
당신의 문서를 AI가 자동으로 분류합니다 PARA 방식 (Projects, Areas, Resources, Archives)으로 스마트하게 정리



예시

2.

2.1



2.4 📊 실시간 대시보드

- PARA 분류 통계 시각화
- 파일 트리 구조 표시
- 최근 활동 로그
- 메타데이터 관리

2.5 🎯 맥락 기반 분류

- 사용자 직업/관심 영역 반영
- 신뢰도 점수 (0-100%)
- 키워드 태그 자동 생성
- 분류 근거 설명

2.6 🤖 지능형 자동화 시스템

- 자동 재분류: 매일/매주 분류 신뢰도가 낮은 문서를 AI가 재검토
- 스마트 아카이빙: 장기간(90일 이상) 수정되지 않은 Projects 문서를 Archives로 이동 제안
- 정기 리포트: 주간/월간 분류 통계 및 인사이트 리포트 생성
- 시스템 모니터링: 백그라운드 작업 상태 및 동기화 무결성 자동 점검
- Celery & Redis: 안정적인 분산 작업 큐 처리

2.7 🔗 MCP 서버 & Obsidian 연동 (v5.0 Phase 1)

- Model Context Protocol (MCP): Claude Desktop과 통합 가능한 MCP 서버 구현
- Obsidian 동기화: 실시간 Vault 파일 감지 및 자동 분류
- 충돌 해결: 3-way 충돌 감지 및 자동 해결 (Rename 전략)
- MCP Tools: `classify_content`, `search_notes`, `get_automation_stats`
- MCP Resources: PARA 카테고리별 파일 리스트 제공

2.8 📊 Next.js 기반 모던 대시보드 (v5.0 Phase 2-3)

- Sync Monitor: Obsidian 연결 상태 및 MCP 서버 상태 실시간 모니터링
- PARA Graph View: React Flow 기반 파일-카테고리 관계 시각화
 - 노드 클릭 인터랙션 (Toast 알림)
 - Deterministic Layout (새로고침 시에도 위치 유지)
 - Zoom/Pan 지원
- Advanced Stats: Recharts 기반 통계 차트
 - Activity Heatmap (GitHub 스타일 연간 활동)
 - Weekly Trend (12주 파일 처리량)
 - PARA Distribution (카테고리별 비중 Pie Chart)
- Mobile Responsive: 데스크탑/모바일 자동 전환 내비게이션
- Accessibility: ARIA 속성 및 스크린 리더 지원

[!NOTE] 개발자용 기능: 대시보드의 실시간 상태 및 그래프 데이터는 WebSocket을 통해 백엔드와 실시간으로 동기화됩니다. (v6.0+)

2.9 🔄 WebSocket 실시간 업데이트 (v6.0 Phase 1)

- 실시간 동기화: Polling 방식 제거, WebSocket 기반 양방향 통신
- 이벤트 기반 업데이트: 파일 분류, 동기화 상태 변경 시 즉시 UI 반영
- 네트워크 최적화: 트래픽 50% 이상 감소
- 연결 관리: 자동 재연결, Heartbeat 메커니즘
- 타입 안전성: TypeScript 기반 이벤트 타이핑

2.0 🔍 Conflict Diff Viewer (v6.0 Phase 2)

- 시각적 비교: Monaco Editor 기반 Side-by-Side Diff 뷰어
- 3가지 해결 옵션: Keep Local / Keep Remote / Keep Both
- Markdown 프리뷰: 충돌 파일의 렌더링된 결과 미리보기
- Syntax Highlighting: 파일 타입별 구문 강조
- 인라인 D

- **개인화 RAG (Personalized RAG):** 사용자별 컨텍스트 및 선호도를 반영한 맞춤형 검색 품질 향상
- **GraphRAG & 지식 맵 시각화:** Vector RAG에 노트 간 연결망(Edge)을 더한 하이브리드 추론 및 **react-force-graph** 기반 지식 그래프 탐색

3. 기술 스택

3.1 Backend

기술	버전	용도
Python	3.11.10	개발 언어
FastAPI	0.120.4	REST API 프레임워크
WebSocket	-	실시간 양방향 통신 (v6.0)
LangChain	1.0.2	AI 체인 관리
Celery	5.4.0	비동기 작업 큐 & 스케줄링
Redis	7.x	메시지 브로커 & 캐시
Flower	2.0.1	Celery 모니터링 대시보드
SQLite	3	메타데이터 저장
Uvicorn	0.38.0	ASGI 서버

3.2 Frontend

기술	버전	용도
Next.js	16.1.1	React 프레임워크 (App Router)
React	19.2.3	UI 라이브러리
TypeScript	5.x	타입 안전성
Tailwind		

기술

OpenAI API GPT-4o 분류, 영역 추천

OpenAI API GPT-4o-m량 작업

OpenAI Embeddings text-embedding-3-small 벡터 임베딩

BM25 Dense (Dense)

BM25 Sparse (Sparse)

BM25 하이브리드 순위 통합 알고리즘

0.11.0 PDF 처리



```
flownote-mvp/
├── requirements.txt          # Python 의존성
├── .env.example              # 환경변수 예시
├── .gitignore
├── backend/                  # FastAPI 백엔드
│   ├── main.py              # FastAPI 메인 앱 (Entrypoint)
│   ├── celery_app/          # Celery 설정
│   │   ├── celery.py        # Celery 인스턴스
│   │   ├── config.py        # Celery 설정
│   │   └── tasks/            # 비동기 작업들 (재분류, 아카이빙 등)
│   ├── mcp/                  # MCP 서버 ✨
│   │   ├── server.py        # MCP 서버 구현
│   │   └── tools/            # MCP Tools (classify, search 등)
│   ├── api/                  # API 엔드포인트
│   │   ├── endpoints/
│   │   │   ├── websocket.py  # WebSocket 엔드포인트 (v6.0) ✨
│   │   │   ├── sync.py      # 동기화 & Diff API (v6.0) ✨
│   │   │   └── ...
│   │   ├── deps.py           # 의존성 (i18n 로케일 추출) ✨
│   │   └── exceptions.py     # 다국어 예외 처리 (v6.0) ✨
│   └── services/             # 비즈니스 로직 (Service Layer)
│       ├── obsidian_sync.py  # Obsidian 동기화 ✨
│       ├── conflict_resolution_service.py # 충돌 해결 ✨
│       └── websocket_manager.py # WebSocket 연결
```

```

├── diff_service.py          # Diff 생성 (v6.0) ✨
├── i18n_service.py         # 다국어 메시지 (v6.0) ✨
├── hybrid_search_service.py # 하이브리드 검색 오케스트레이터 (v7.0) ✨
├── search_cache_service.py # Redis 검색 결과 캐싱 (v7.0) ✨
├── core/
│   └── config.py           # 핵심 설정 (v6.0) ✨
│                           # 애플리케이션 설정
├── embedding.py            # 임베딩 생성
├── faiss_search.py         # FAISS 벡터 검색 (save/load 영속화 포
함) ✨
├── bm25_search.py         # BM25 키워드 검색 (save/load 영속화 포
함) ✨
├── hybrid_search.py       # FAISS+BM25 RRF 하이브리드 엔진 (v7.0)
✨
├── classifier/
│   └── ...                # PARA 분류 로직
├── scripts/
│   └── bootstrap_index.py # 유틸리티 스크립트
(v7.0) ✨ # Obsidian Vault 초기 인덱싱 CLI
├── web_ui/
│   └── src/
│       ├── app/
│       │   ├── [locale]/
│       │   │   ├── page.tsx          # 다국어 라우팅 (v6.0) ✨
│       │   │   ├── graph/page.tsx    # Dashboard
│       │   │   ├── stats/page.tsx    # Graph View
│       │   │   └── search/page.tsx    # Statistics
│       │   └── not-found.tsx          # 하이브리드 검색 페이지 (v7.0) ✨
│       ├── components/
│       │   ├── dashboard/
│       │   │   └── SyncMonitor.tsx    # 404 페이지 (i18n) ✨
│       │   ├── para/GraphView.tsx    # React 컴포넌트
│       │   ├── search/
│       │   │   └── HybridSearch.tsx   # WebSocket 기반 (v6.0) ✨
│       │   ├── conflict/
│       │   │   ├── DiffViewer.tsx    # Graph View
│       │   │   └── ConflictResolver.tsx
│       │   └── layout/
│       │       └── LanguageSwitcher.tsx # 하이브리드 검색 UI (v7.0) ✨
│       ├── i18n/
│       │   ├── config.ts             # Conflict Diff Viewer (v6.0) ✨
│       │   └── lib/
│       │       └── searchApi.ts       # 언어 전환 (v6.0) ✨
│       └── locales/
│           ├── ko.json               # 다국어 설정 (v6.0) ✨
│           └── en.json
│       ├── hooks/
│       │   └── useWebSocket.ts       # Custom Hooks
│       ├── config/
│       │   └── middleware.ts         # WebSocket Hook (v6.0) ✨
│       └── ...                      # 설정 파일
│                                   # next-intl 미들웨어 (v6.0) ✨

```

└─ package.json	
└─ tests/	
└─ unit/	# 단위 테스트
└─ e2e/	
└─ test_rag_search_quality.py	# E2E 검색 품질 측정 (v7.0) ✨
└─ performance/	
└─ benchmark_rag.py	# 대용량 성능 벤치마크 (v7.0) ✨
└─ data/	# 데이터 저장소
└─ docs/	# 문서
└─ AR/	# 아카이브 (지난 페이지 문서)
└─ v4.0/	# v4.0 (Refactoring)
└─ v5.0/	# v5.0 (MCP, Frontend 등)
└─ v6.0/	# v6.0 (WebSocket, Diff, i18n)
└─ v7.0/	# v7.0 (Hybrid RAG)
└─ v8.0/	# v8.0 (Data Flywheel)
└─ v9.0/	# v9.0 (Adaptive Intelligence) ✨
└─ P/	# 진행 중인 페이지 문서
└─ A/	# 분석 및 명세 (Practices, Specs)
└─ [TODO.md] (docs/A/TODO.md)	# 기술 부채 및 잔여
작업 추적 문서 (우선순위 기반)	
└─ [v10_ideas.md] (docs/A/v10_ideas.md)	# 차기 마일스톤
기획 초기 뼈대 문서	
└─ R/	# 리소스 (Troubleshooting 등)
└─ README.md	# 본 문서 (한국어)
└─ README_EN.md	# 영문 문서

5. 🪄 테스트 및 품질 관리

FlowNote는 엄격한 테스트와 품질 관리를 통해 안정성을 보장합니다.

5.1 테스트 실행

[!IMPORTANT] ~~개발자 전용: 아래 명령어는 시스템의 안정성을 검증하기 위한 테스트 코드로, 일반 사용자는 실행할 필요가 없습니다.~~

```
# 전체 테스트 실행
pytest

# 커버리지 리포트 생성
pytest --cov=backend --cov-report=term-missing
```

5.2 테스트 커버리지 (v7.0 Phase 2 기준)

모듈	커버리지	비고
전체	55%+	주요 로직 위주 테스트

모듈	커버리지	비고
<code>util.py</code> (Celery Tasks)	80%+	자동화 태스크
<code>parallel_processor.py</code>	100%	병렬 처리
<code>classification_service.py</code>	89%	핵심 분류 로직
<code>hybrid_search_service.py</code>	90%+	하이브리드 검색 서비스 (v7.0)
<code>search_cache_service.py</code>	85%+	Redis 캐시 레이어 (v7.0)

5.3 E2E 검색 품질 측정 (v7.0)

```
# 초기 인덱스 구축 (Obsidian Vault 전체 인덱싱)
# ⚠️ --clear: 기존 인덱스를 완전히 삭제하고 재구축합니다.
#           프로덕션 환경에서는 신중하게 사용하세요 (복구 불가).
python scripts/bootstrap_index.py --vault /path/to/your/vault --clear

# ✅ 인덱스 업데이트만 필요할 경우: --clear 없이 실행 (기존 인덱스에 추가)
python scripts/bootstrap_index.py --vault /path/to/your/vault

# E2E 검색 품질 실측 (OpenAI API 연동 필요)
pytest tests/e2e/test_rag_search_quality.py -s -v

# 대용량 성능 벤치마크 (1,000개 이상 문서)
pytest tests/performance/benchmark_rag.py -s -v
```

📁 인덱스 저장 위치: `backend/config/__init__.py`의 `PathConfig.FAISS_INDEX_DIR` 환경변수로 제어됩니다 (기본값: `data/indices/`).

지표	측정값	측정 조건
Precision	0.75	테스트 Vault (~20개 문서, 5개 쿼리셋, <code>alpha=0.5</code>)
Recall	0.92	<code>filter_expansion_factor=2.0</code> 튜닝 후 동일 조건

■ 참고: 위 수치는 소규모 테스트 데이터셋 기준입니다. 실제 Vault 규모와 쿼리 특성에 따라 결과가 달라질 수 있으며, `tests/performance/benchmark_rag.py`로 본인 환경에서 직접 측정하는 것을 권장합니다.

6. 🚀 설치 및 실행

6.1 사전 요구사항

- **Python:** 3.11+
- **Node.js:** 18+
- **OpenAI API Key:** platform.openai.com
- **Redis Server:** 6.2+ (Celery 브로커용)

6.2 설치 방법

```
git clone https://github.com/jjaayy2222/flownote-mvp.git
```

설치 및 활성화

```
python3 -m venv venv
```

```
venv/bin/activate
```

(macOS)

```
brew install redis
```

```
brew start redis
```

```
pip install -r requirements.txt
```

패키지 설치

변경

```
source .env
```

OpenAI API 키 및 REDIS

실행 가이드

서버를 사용하려면 4개의 터미널 (+ MCP 서버 사용)이 필요합니다.

1. 터미널 실행 (Terminal 1)

루트 디렉토리에서

macOS:

```
venv/bin/activate # 또는 python3 -m venv <your-env>
```

Windows:

```
venv\Scripts\activate
```

```
python -m uvicorn backend.main:app --reload
```

```
→ http://127.0.0.1:8000
```

2. Next.js Frontend 실행 (Terminal 2)

```
cd web_ui
```

```
npm run dev
```

```
# → http://localhost:3000
```

```
# 한국어: http://localhost:3000/ko
```

```
# 영어: http://localhost:3000/en
```

Worker & Beat 실행 (Terminal)

```
celery -A backend.celery worker --beat --loglevel=info
```

```
celery -A backend.celery flower --port=5555  
# → http://localhost:5555
```

```
# Claude Desktop 연동 시  
python -m backend.mcp.server  
# 또는 Claude Desktop 설정에서 자동 시작
```

7. 📖

Tab 1: 온보딩

Tab 2: 파일 분류

7.6 Step 6: 자동화 모니터링

- 1. Flower 대시보드 접속
- 2. Tasks 탭에서 자동 재분류/리포트 생성 작업 확인
- 3. System 탭에서 워커 상태 확인

7.7 Step 7: 하이브리드 RAG 검색 사용 (v7.0)

[!TIP] 초기 설정: 하이브리드 검색을 처음 사용하기 전, 한 번의 인덱싱 작업이 필요합니다 (아래 참조).

- 1. 초기 인덱스 구축 (최초 1회)

```
# Obsidian Vault 전체를 FAISS + BM25 인덱스로 일괄 색인
python scripts/bootstrap_index.py --vault /path/to/your/vault
```

- 2. 대시보드 또는 /search 페이지에서 검색어 입력
- 3. Hybrid Search 탭에서 alpha (Dense/Sparse 가중치), k (결과 수), PARA 카테고리 필터 설정
- 4. 검색 결과의 Score 및 응답 지연시간(Latency) 확인

7.8 Step 8: CLI 사용

```
# 단일 파일 분류
python -m backend.cli classify "path/to/file.txt" [user_id]
```

8. 개발 히스토리

Issue별 개발 진행도

Issue	완료일	핵심 기능	상태
[#1-10]	~11/11	Phase 1-2 (MVP)	✓
[#10.4]	12/16	Celery 자동화 & 스케줄링	✓
[#10.11]	02/04	v6.0 Phase 3 (i18n)	✓
[#11.2.12]	03/02	v7.0 Phase 2 (Hybrid RAG)	✓
[#11.3.13]	03/12	v7.0 Phase 2-3 (Hybrid RAG & AI Assistant)	✓
[#867]	03/25	v8.0 Phase 1-3 (Advanced RAG Eval & Performance)	✓
[#1045]	06/13	v9.0 Phase 1-4 (Adaptive Intelligence & GraphRAG)	✓ 완료

주요 커밋 히스토리

- v5.0 - MCP 서버, Next.js 대시보드, Graph View
- v6.0 Phase 1 - WebSocket 실시간 업데이트

- v6.0 Phase 2 - Conflict Diff Viewer
- v6.0 Phase 3 - 다국어 지원 (i18n)
- v7.0 - 하이브리드 RAG 검색 엔진 및 AI 어시스턴트
- v8.0 - RAG 품질 평가 파이프라인 및 성능 최적화 ✓
- v9.0 - 자가 적응 지능(Adaptive Intelligence) 및 GraphRAG 도입 ✓

9. 🗺️ 로드맵

✓ 완료된 기능 (v5.0)

- ☒ 스마트 온보딩 & PARA 분류
- ☒ Celery 기반 비동기 작업 큐
- ☒ 정기 자동 재분류 및 아카이빙 스케줄러
- ☒ Flower 모니터링 통합
- ☒ MCP 서버 구현 ✨
- ☒ Obsidian 동기화 및 충돌 해결 ✨
- ☒ Next.js 기반 모던 대시보드 ✨
- ☒ PARA Graph View & Advanced Stats ✨
- ☒

✓ 완료

- ☒ Phase 1: RESEARCHER 기반 업데이트 ✨
 - ☒ Polling 방식 제거



• ☒ P

• ☐ C

• ☐ C

• ☐ C

• ☐ C

• ☐ C

• ☐ C

• ☐ C

• ☐ C

• ☐ C

• ☐ C

• ☐ C

• ☐ C

• ☐ C

• ☐ C

• ☐ C

• ☐ C

• ☐ C

• ☐ C

• ☐ C

• ☐ C

• ☐ C

• ☐ C

• ☐ C

• ☐ C

• ☐ C

• ☐ C

• ☐ C

• ☐ C

• ☐ C

• ☐ C

• ☐ C

• ☐ C

• ☐ C

• ☐ C

• ☐ C

• ☐ C

A: 네, Celery 메시지 브로커 및 v7.0의 검색 결과 캐싱(**SearchCacheService**)에도 Redis를 사용하므로 반드시 실행되어 있어야 합니다.

Q2. 자동화 작업은 언제 실행되나요?

A: **backend/celery_app/config.py**에 정의된 스케줄에 따릅니다. (예: 재분류-매일 00:00)

Q3. 하이브리드 검색을 사용하려면 초기 설정이 필요한가요?

A: 네, 서버 최초 실행 전에 **scripts/bootstrap_index.py**로 Obsidian Vault를 인덱싱해야 합니다. 이후 서버 재시작 시에는 인덱스가 자동으로 디스크에서 로드됩니다.

Q4. **alpha** 파라미터는 무엇인가요?

A: FAISS(Dense, 의미 기반)와 BM25(Sparse, 키워드 기반) 검색의 가중치 비율입니다. **alpha=1.0**이면 FAISS만, **alpha=0.0**이면 BM25만 사용합니다. 기본값은 **0.5** (균등 혼합)입니다.

Q5. 검색 결과가 느릴 때 어떻게 하나요?

A: Redis 캐시가 적중되면 응답이 즉시 반환됩니다. 캐시 미스 시 첫 검색은 임베딩 연산이 포함되어 느릴 수 있습니다. **tests/performance/benchmark_rag.py**로 성능을 측정해 볼 수 있습니다.

11. 🤝 기여하기

Issues 제보 및 PR 환영합니다.

12. 📄 라이선스

MIT License

13. 👤 개발자

Jay (@jjaayy2222)